

Day 1: Foundations of Parallel Computing & Concurrency in Python

Duration: 4 hours (theory + live demos)

Prerequisites: Basic Python programming

Demos folder: [day1/demo/](#)

Table of Contents

1. [Data Serialization](#)
 2. [Threads and Processes — Concurrency Fundamentals](#)
 3. [Getting Started with Parallel Computing](#)
 4. [Parallel Performance — Pool, Speedup & Amdahl's Law](#)
-

1. Data Serialization

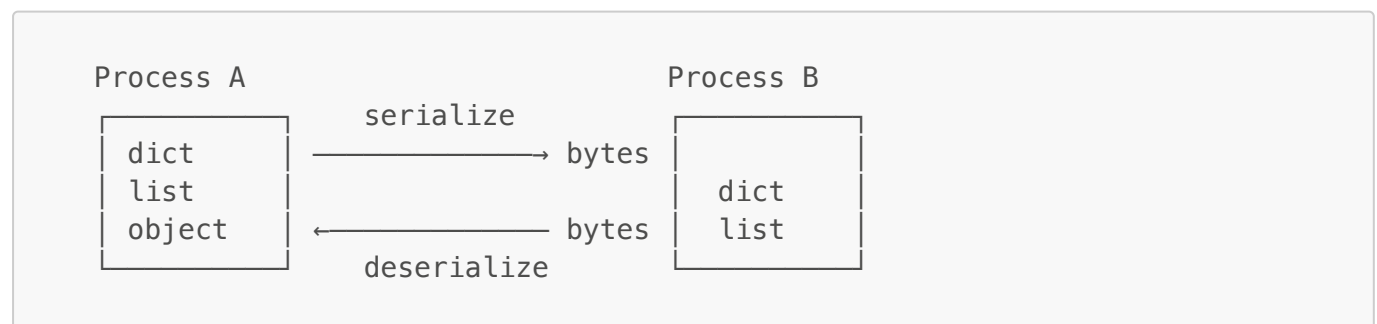
Why Start Here?

Before we write parallel programs, we need to understand **how data moves** between processes. When you run two separate Python processes, they do **not** share memory. So how does one process send a Python dictionary to another?

The answer is **serialization** — converting an in-memory Python object into a sequence of bytes (or text) that can be:

- Written to a file
- Sent over a network socket
- Passed through a pipe between processes

The receiving end **deserializes** (decodes) those bytes back into a live Python object.



Python gives us two built-in serialization modules: **JSON** and **Pickle**.

1.1 JSON Serialization

JSON (JavaScript Object Notation) is a lightweight, text-based, human-readable format for data exchange. It is the lingua franca of web APIs and configuration files.

Supported Python Types

Python Type	JSON Equivalent
<code>dict</code>	Object <code>{}</code>
<code>list</code>	Array <code>[]</code>
<code>str</code>	String <code>""</code>
<code>int</code>	Number
<code>float</code>	Number
<code>True/False</code>	<code>true/false</code>
<code>None</code>	<code>null</code>

Core API

Function	Direction
<code>json.dumps()</code>	Python object → JSON string
<code>json.loads()</code>	JSON string → Python object
<code>json.dump()</code>	Python object → JSON file
<code>json.load()</code>	JSON file → Python object

Limitations

- **No support** for `set`, `tuple` (silently becomes a list), `bytes`, `datetime`, or custom class instances.
 - This is by design — JSON is meant to be **cross-language** (Java, C++, JavaScript, Go, etc. can all read it).
-

1.2 Pickle Serialization

Pickle is Python's native binary serialization protocol. Unlike JSON, it can serialize **almost any** Python object — including custom classes, tuples, sets, functions, and more.

Core API

Function	Direction
<code>pickle.dumps()</code>	Python object → bytes
<code>pickle.loads()</code>	bytes → Python object
<code>pickle.dump()</code>	Python object → binary file
<code>pickle.load()</code>	binary file → Python object

Pickle vs JSON — Quick Comparison

Feature	JSON	Pickle
Format	Text (UTF-8)	Binary
Human-readable	Yes	No
Custom objects	No	Yes
Tuples / Sets	No	Yes
Cross-language	Yes	Python only
Security	Safe	⚠️ Unsafe

⚠️ **Security Warning:** Never unpickle data received from an untrusted or unauthenticated source. A malicious pickle payload can execute arbitrary code during deserialization.

Why Pickle Matters for Parallel Python

Python's `multiprocessing` module uses **pickle internally** to:

- Send function arguments to worker processes (via `Pool.map()`)
- Return results from workers back to the parent
- Transfer objects through `Queue` and `Pipe`

If an object **cannot be pickled**, it **cannot be sent** to another process. Understanding this saves hours of debugging "can't pickle" errors later.

🖥️ Demo Time

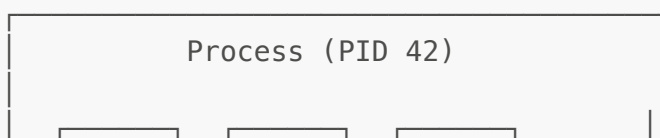
Now, let's see these theoretical topics in action.

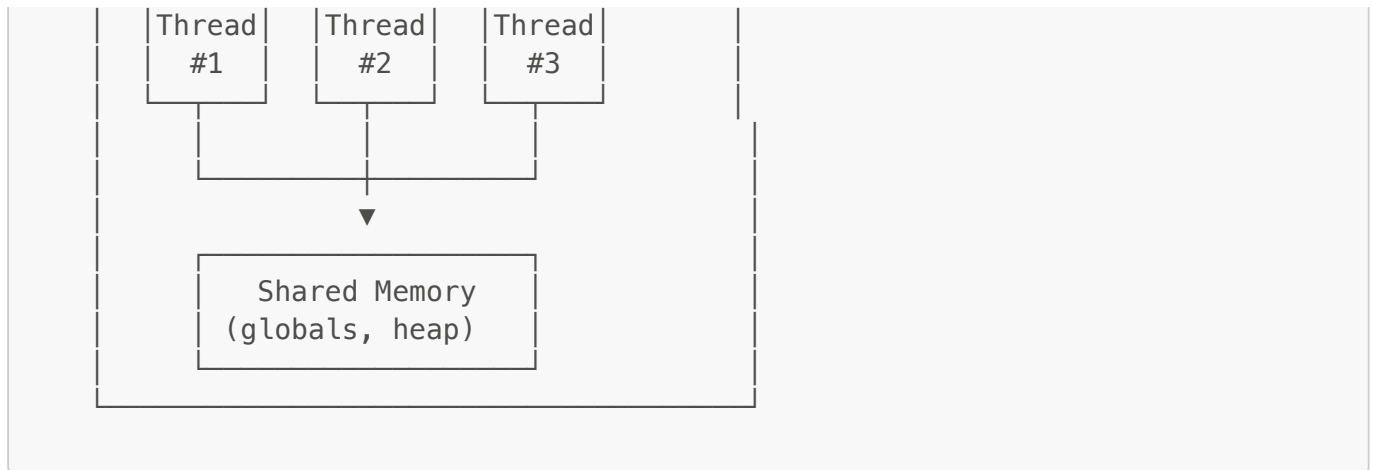
- `01_json_serialization.py` — Serialize and deserialize Python data structures with JSON.
- `02_pickle_serialization.py` — Pickle complex objects including custom classes, and see the comparison table live.

2. Threads and Processes — Concurrency Fundamentals

What is a Thread?

A **thread** is a lightweight unit of execution that exists **within** a process. Multiple threads share the **same memory space** — they can read and write the same variables, lists, and dictionaries directly.





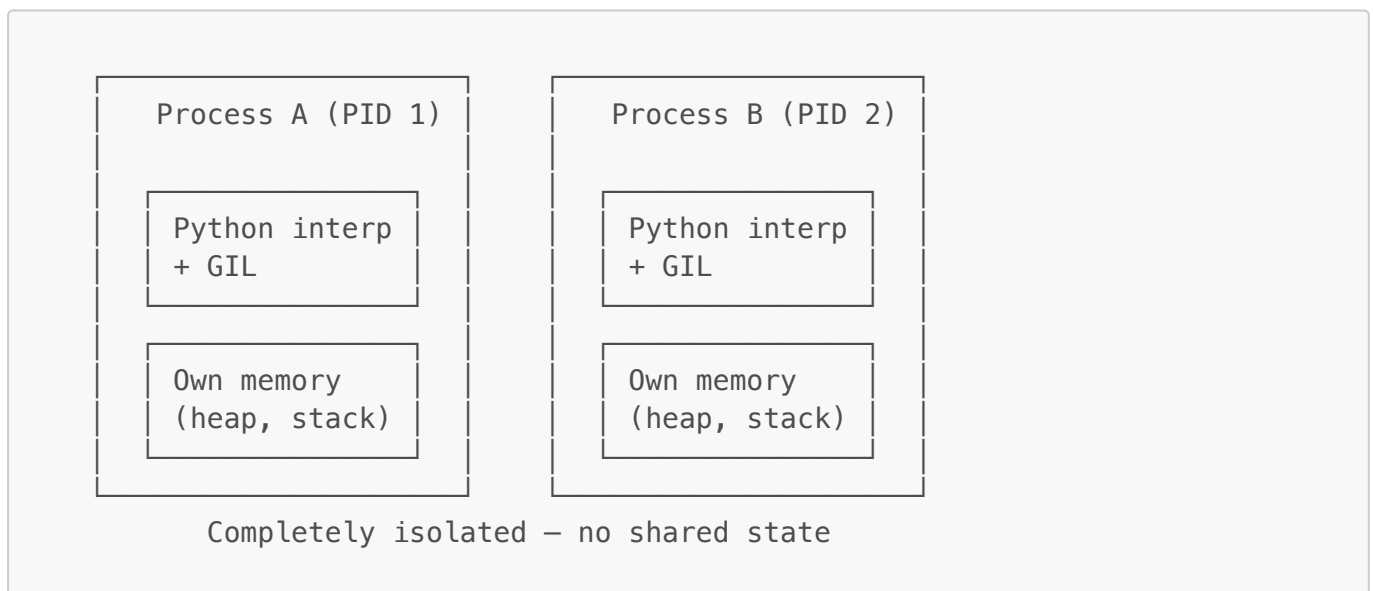
Key observations:

- All threads share the **same PID** (Process ID).
- Each thread has a unique **TID** (Thread Identifier) — `threading.get_ident()`.
- Threads can directly append to a shared list, modify a shared dict, etc.

What is a Process?

A **process** is an independent instance of a running program. Each process has:

- Its **own memory space** (heap, stack, global variables)
- Its **own Python interpreter** (including its own GIL — more on this soon)
- A unique **Process ID (PID)** assigned by the operating system



If a child process modifies a global variable, **the parent never sees the change** — the child was working on its own private copy all along.

Threads vs Processes — Key Differences

Property	Thread	Process
Memory	Shared (same address space)	Isolated (separate spaces)

Property	Thread	Process
Creation cost	Low (lightweight)	High (fork/spawn)
Communication	Direct variable access	IPC (pipes, queues, etc.)
GIL impact	Contend for one GIL	Each has its own GIL
Crash isolation	One crash kills all threads	One crash $\neq$ other crashes

The Global Interpreter Lock (GIL)

The **GIL** is a mutex built into CPython (the standard Python implementation). It allows **only one thread** to execute Python bytecode at any given time, even on a multi-core machine.

Without GIL (what you'd expect):

```
Thread 1 [█] RUNNING (Core 1)
Thread 2 [█] RUNNING (Core 2)
Thread 3 [█] RUNNING (Core 3)
Thread 4 [█] RUNNING (Core 4)
```

With GIL (reality in CPython):

```
Thread 1 [█] RUNNING (Core 1)
Thread 2 [░░░] WAITING (GIL)
Thread 3 [░░░] WAITING (GIL)
Thread 4 [░░░] WAITING (GIL)
```

Why Does the GIL Exist?

1. **Memory management safety** — CPython uses reference counting for garbage collection. Without the GIL, every `Py_INCREF` / `Py_DECREF` would need its own lock, which would be slower overall.
2. **C extension compatibility** — Many C extensions assume exclusive access to Python objects.
3. **Simplicity** — A single global lock is simpler than fine-grained locking.

The GIL's Impact: CPU-Bound vs I/O-Bound

The GIL's impact depends entirely on the **type of work** your threads do:

CPU-Bound Tasks (computation-heavy):

```
Thread 1: [█] compute [█] [wait] [█] compute [█] [wait]...
Thread 2: [wait] [█] compute [█] [wait] [█] compute [█]...
           ↑
       Only ONE computes at a time → threads ≈ sequential
```

- **Threads:** No speedup. The GIL serializes execution.
- **Processes:** Real speedup! Each process has its own GIL.

I/O-Bound Tasks (network, disk, sleep):

```
Thread 1: [request].....[response] [process]
Thread 2:      [request].....[response] [process]
```

Thread 3: [request].....[response] [process]

↑
GIL is RELEASED during I/O waits → real speedup ✓

- **Threads:** Great speedup! The GIL is **released** during I/O waits.
- **Processes:** Also work, but threads are cheaper and simpler.

Summary: When to Use What

Workload Type	Use Threads?	Use Processes?
CPU-bound	✗ No speedup	✓ Real parallelism
I/O-bound	✓ Great speedup	✓ Works (heavier)
Mixed	Consider both	Processes safer

Race Conditions and Locks

The Problem: Race Conditions

Because threads share memory, one thread can modify a variable that another thread is reading at the same time. When the outcome depends on the **timing of thread execution**, you have a **race condition**:

```
counter = 0

def increment():
    global counter
    for _ in range(100_000):
        counter += 1    # NOT atomic!
        # Internally: temp = counter → temp += 1 → counter = temp
        # Another thread can interleave between these steps!
```

If two threads run `increment()`, the final value of `counter` is **unpredictable** — it could be anything between 100,000 and 200,000.

The Fix: `threading.Lock`

A Lock ensures only **one thread** can enter the protected section at a time:

```
lock = threading.Lock()

def safe_increment():
    global counter
    for _ in range(100_000):
        with lock:          # only one thread enters this block at a time
            counter += 1    # now it's safe
```

Sharing State Between Processes

Since processes have **isolated memory**, you need explicit mechanisms to share data:

Mechanism	Use Case
<code>multiprocessing.Value</code>	A single shared variable (int, float, etc.)
<code>multiprocessing.Array</code>	A shared array of fixed type
<code>multiprocessing.Queue</code>	Message passing between processes
<code>multiprocessing.Pipe</code>	Two-way communication channel
<code>multiprocessing.Manager</code>	Shared dicts, lists (slower, via proxy)

Example with `multiprocessing.Value`:

```
shared_counter = multiprocessing.Value('i', 0)  # 'i' = int, initial = 0

def increment(counter, n):
    for _ in range(n):
        with counter.get_lock():  # built-in lock
            counter.value += 1
```

Demo Time

Now, let's see these theoretical topics in action.

- `03_threads_vs_processes.py` — Four progressive demos in one file:
 1. `demo_threads_basics()` — Threads share PID and memory
 2. `demo_processes_basics()` — Processes have isolated memory; fix with `Value`
 3. `demo_gil()` — CPU-bound vs I/O-bound benchmark (sequential, threaded, multiprocessing)
 4. `demo_race_condition()` — Race condition live, then fixed with `Lock`

3. Getting Started with Parallel Computing

Why Parallel Computing?

The era of "just wait for faster CPUs" is over. Since around 2005, CPU clock speeds have plateaued at ~3–5 GHz due to physical limits (power dissipation, heat). Instead, hardware advances now come as **more cores**, not faster cores:

Clock Speed Growth

1990: 40 MHz

Core Count Growth

1990: 1 core

2000: 1 GHz	2005: 2 cores
2005: 3.8 GHz	2010: 4–8 cores
2010: 3.5 GHz ← plateau	2020: 16–64 cores
2024: 3.5 GHz ← same	2024: 64–128+ cores

To exploit modern hardware, software **must** run in parallel.

The Parallel Computing Memory Architecture

Computer architectures are classified by **Flynn's Taxonomy** based on how they handle **instruction streams** and **data streams**:

		Data Streams	
		Single	Multiple
Instruction Streams:	Single	SISD	SIMD
	Multiple	MISD	MIMD

SISD — Single Instruction, Single Data

- Traditional **sequential** (von Neumann) computer.
- One instruction operates on one piece of data at a time.
- Example: a simple single-core CPU executing a program line by line.

SIMD — Single Instruction, Multiple Data

- **One instruction** is applied to **many data elements** simultaneously.
- Example: GPU cores, Intel SSE/AVX vector instructions.
- Ideal for: image processing, matrix operations, signal processing.

SIMD Example: Add two arrays

A = [1, 2, 3, 4]

B = [5, 6, 7, 8]

|
Single ADD instruction

C = [6, 8, 10, 12] ← all four additions happen simultaneously

MISD — Multiple Instruction, Single Data

- **Multiple instructions** operate on the **same data stream**.
- Rare in practice; used in fault-tolerant systems (e.g., redundant flight computers processing the same sensor data through different algorithms).

MIMD — Multiple Instruction, Multiple Data

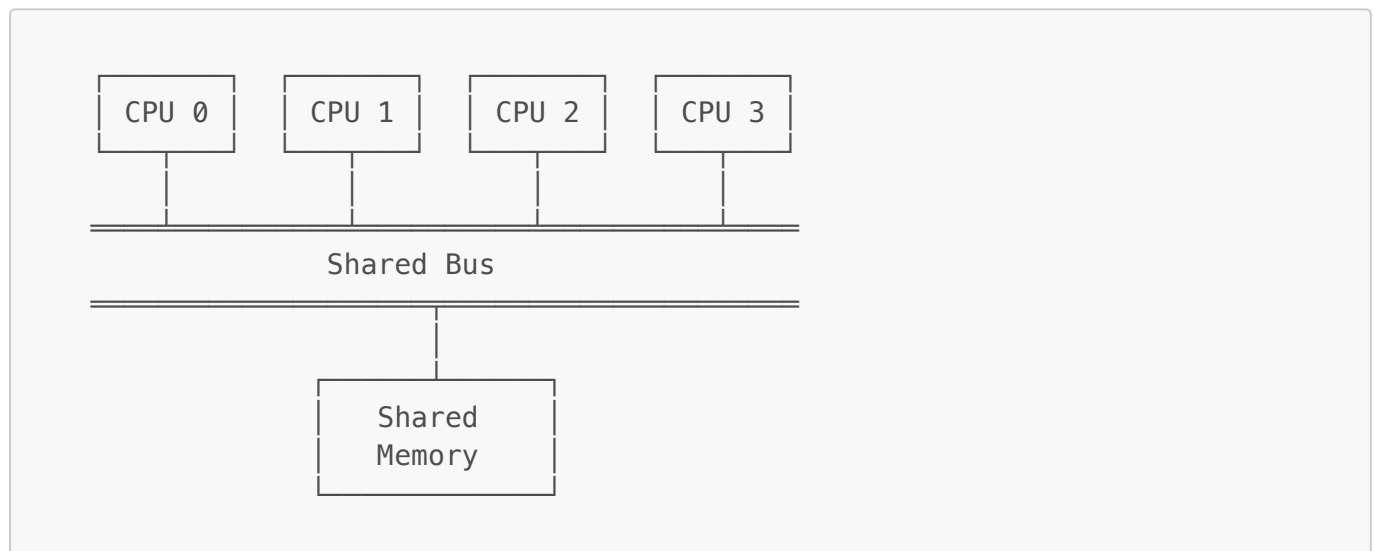
- **Multiple processors** execute **different instructions** on **different data** simultaneously.
- This is the **most common** parallel architecture today.
- Examples: multi-core CPUs, clusters of workstations, cloud computing.

Memory Organization

How processors **access memory** fundamentally shapes how parallel programs are designed:

Shared Memory

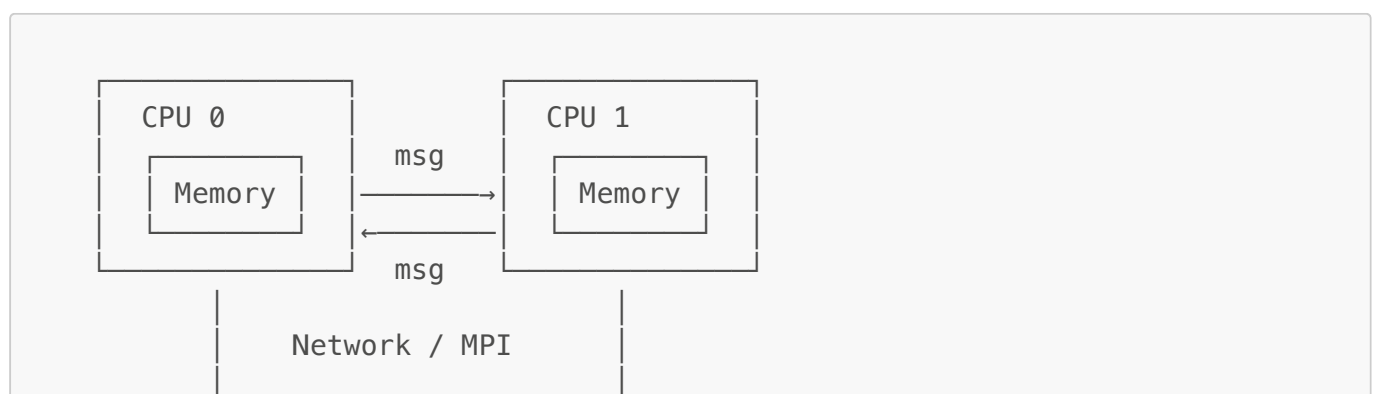
All processors access a **common memory space** through a shared bus or interconnect:

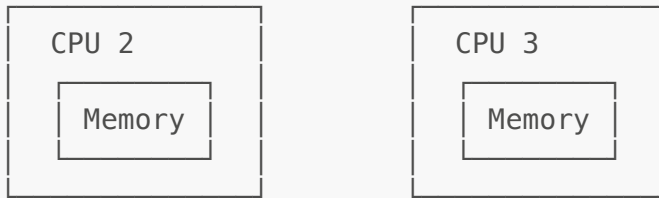


- **Pros:** Easy to program (just read/write shared variables).
- **Cons:** Bus contention, cache coherency overhead, doesn't scale beyond ~64–128 cores.

Distributed Memory

Each processor has its **own private memory**. Processors communicate by **sending messages** over a network:

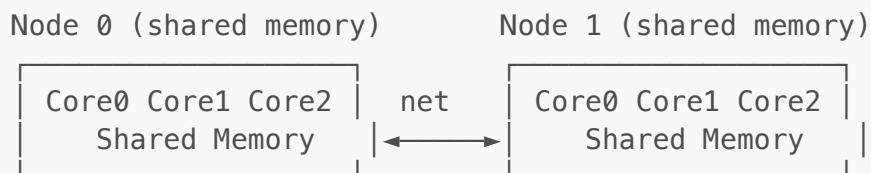




- **Pros:** Scales to thousands of nodes, no cache coherency issues.
- **Cons:** Programmer must explicitly manage all data movement (MPI).

Hybrid: Clusters of Multi-Core Nodes

Modern HPC systems combine **both**: shared memory within each node, distributed memory across nodes.



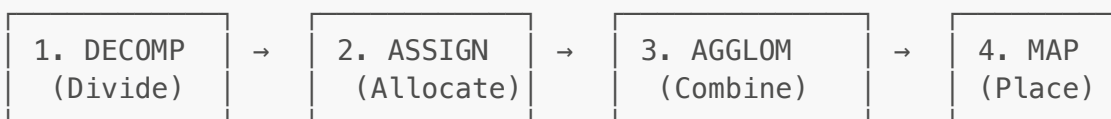
- Within a node → threads or multiprocessing (shared memory)
- Across nodes → MPI message passing (covered on Day 3)

Parallel Programming Models

Model	Memory Model	Communication	Python Module
Shared memory	Shared	Direct access	<code>threading</code>
Multithread	Shared	Shared variables	<code>threading</code>
Message passing	Distributed	Send/Receive	<code>mpi4py</code> (Day 3)
Data parallel	Either	Implicit	<code>Pool.map()</code> , GPU

How to Design a Parallel Program

Designing a parallel program follows a four-step methodology called **Foster's Design Methodology**:



Step 1: Task Decomposition

Break the problem into **small, independent tasks** that can run concurrently.

Problem: Process 1000 images

Decomposition:

Task 0: process image[0]

Task 1: process image[1]

...

Task 999: process image[999]

Step 2: Task Assignment

Decide which tasks go to which processor/thread. The goal is **balanced load** — each processor should do roughly equal work.

Step 3: Agglomeration

Combine small tasks into **larger chunks** to reduce overhead. Sending 1000 one-image tasks has more overhead than sending 10 hundred-image tasks.

Before agglomeration:

[t0][t1][t2]...[t999]

1000 tasks

After agglomeration:

[t0-t99][t100-t199]...[t900-t999]

10 chunks

Step 4: Mapping

Assign chunks to **physical processors**. Strategies include:

- **Static mapping:** Pre-assign chunks (simple, good for uniform tasks).
- **Dynamic mapping:** Workers pull tasks from a queue (good for non-uniform tasks).

Dynamic mapping variants:

Strategy	Description
Manager/Worker	One manager distributes tasks to workers on request
Hierarchical Manager/Worker	Multiple levels of managers (for very large systems)
Decentralized	Workers steal tasks from each other

4. Parallel Performance — Pool, Speedup & Amdahl's Law

Data Parallelism with `Pool.map()`

When you have a **list of inputs** and want to apply the **same function** to each one, `multiprocessing.Pool` is the easiest approach:

```

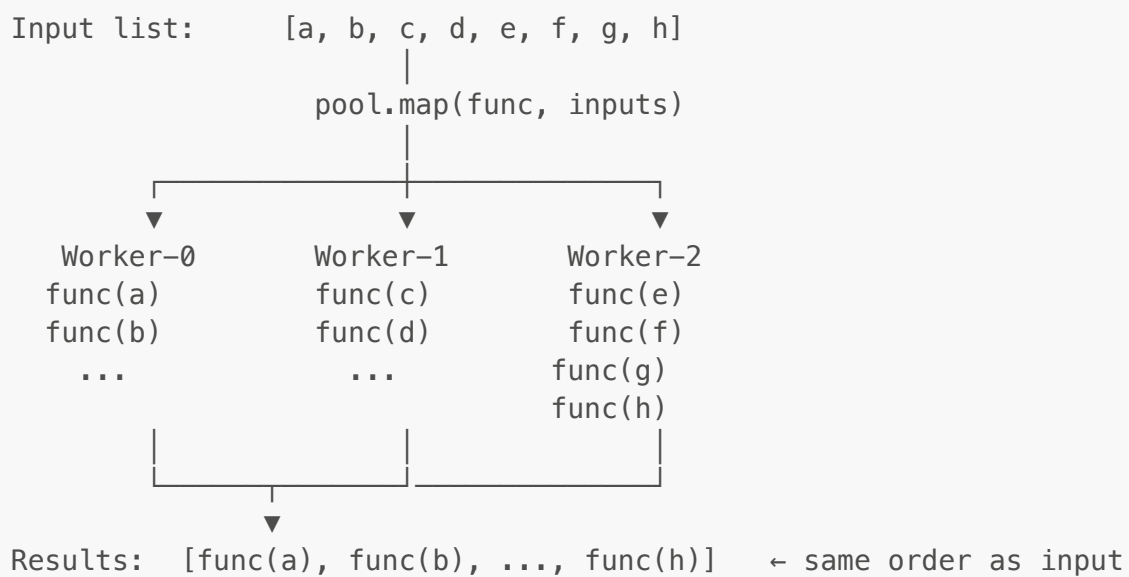
from multiprocessing import Pool

def square(x):
    return x * x

with Pool(processes=4) as pool:
    results = pool.map(square, [1, 2, 3, 4, 5])
# results = [1, 4, 9, 16, 25]

```

How `Pool.map()` works internally:



The pool automatically:

1. Creates N worker processes.
2. Distributes chunks of the input to workers.
3. Collects and **orders** the results.
4. Returns them as a list in the **same order** as the input.

How Do We Know If Our Parallel Program Is Good?

Three key metrics tell us how well we parallelized:

4.1 Speedup

Speedup measures how much faster the parallel version is compared to the sequential version:

$$S(N) = \frac{T_{\text{sequential}}}{T_{\text{parallel}}(N)}$$

where N is the number of processors.

Speedup Value	Meaning
$S(N) = 1$	No improvement (same as sequential)
$S(N) = N$	Linear speedup (ideal / perfect)
$S(N) > N$	Super-linear (rare, due to cache effects)
$S(N) < 1$	Slowdown (overhead > benefit)

4.2 Efficiency

Efficiency measures how well we're utilizing the processors:

$$E(N) = \frac{S(N)}{N} = \frac{T_{\text{sequential}}}{N \times T_{\text{parallel}}(N)}$$

Efficiency	Meaning
$E = 1.0$ (100%)	Every processor is busy 100% of the time
$E = 0.5$ (50%)	Half the processor time is wasted
$E \rightarrow 0$	Severe parallelization overhead

4.3 Amdahl's Law

Amdahl's Law gives the **theoretical maximum speedup** for a program with a serial (non-parallelizable) fraction S :

$$\text{Speedup}(N) = \frac{1}{S + \frac{1-S}{N}}$$

where:

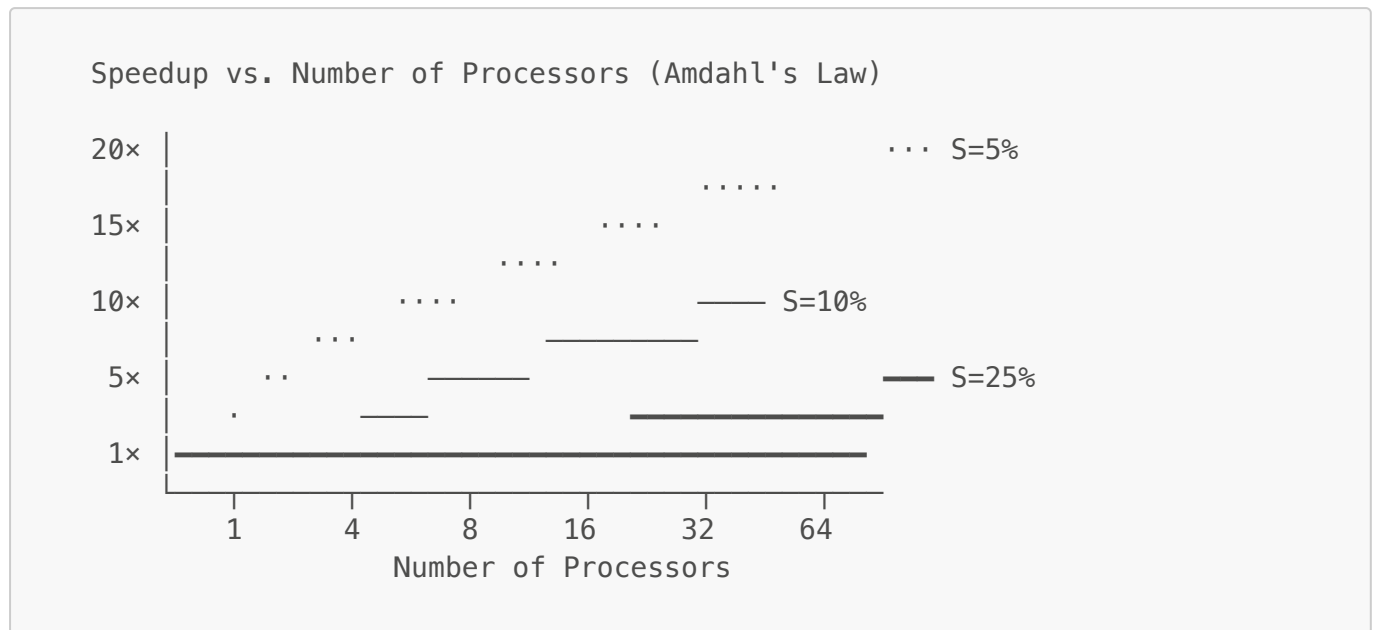
- S = fraction of the program that is strictly serial ($0 \leq S \leq 1$)
- N = number of processors
- $(1 - S)$ = parallelizable fraction

Key insight: Even with **infinite processors** ($N \rightarrow \infty$):

$$\text{Max Speedup} = \frac{1}{S}$$

Serial Fraction (S)	Max Speedup ($N \rightarrow \infty$)
50%	2×
25%	4×
10%	10×
5%	20×
1%	100×

A program with just **5% serial code** can never exceed **20× speedup**, no matter how many processors you throw at it. This is why optimizing the serial portions of code matters so much.



4.4 Gustafson's Law — A More Optimistic View

Amdahl's Law assumes a **fixed problem size**. But in practice, when we get more processors, we often **increase the problem size** to get better results (higher resolution, more data points, larger simulations).

Gustafson's Law measures **scaled speedup**:

$$S_{\text{scaled}}(N) = N - s \times (N - 1)$$

where s is the serial fraction of the **parallel** workload.

Interpretation	Amdahl	Gustafson
Problem size	Fixed	Scales with N
More processors means...	Diminishing returns	Near-linear speedup
Outlook	Pessimistic	Optimistic
When to use	"How fast can I solve THIS?"	"How BIG can I solve in the same time?"

Putting It All Together: Python in a Parallel World

Here's how Python's tools map to the concepts we've learned:

Concept	Python Tool
Process creation	<code>multiprocessing.Process</code>
Process pool	<code>multiprocessing.Pool</code>
Thread creation	<code>threading.Thread</code>

Concept	Python Tool
Thread synchronization	<code>threading.Lock</code> , <code>Event</code> , <code>Semaphore</code>
Inter-thread communication	<code>queue.Queue</code>
Inter-process communication	<code>multiprocessing.Queue</code> , <code>Pipe</code> , <code>Value</code>
CPU-bound parallelism	<code>multiprocessing</code> (bypass GIL)
I/O-bound concurrency	<code>threading</code> (GIL released during I/O)
Data parallelism	<code>Pool.map()</code> , <code>Pool.starmap()</code>
Distributed parallelism	<code>mpi4py</code> (Day 3), <code>Celery</code> (Day 4)
GPU parallelism	<code>PyCUDA</code> , <code>PyOpenCL</code> (Day 5)

Demo Time

Now, let's see these theoretical topics in action.

- `04_parallel_performance.py` — Three progressive demos in one file:
 1. `demo_pool_map()` — `Pool.map()` sequential vs parallel with real speedup
 2. `demo_speedup_efficiency()` — Measure Speedup and Efficiency across 1, 2, 4, ... N cores
 3. `demo_amdahl_gustafson()` — Compare actual speedup against Amdahl's Law and Gustafson's Law

Day 1 Summary

What We Learned	Key Takeaway
Serialization	JSON (cross-language, text) vs Pickle (Python-only, binary, supports all types). Pickle is used internally by <code>multiprocessing</code> .
Threads	Shared memory, one GIL → great for I/O-bound work, limited for CPU-bound
Processes	Independent memory, own GIL → true parallelism for CPU-bound work
GIL	Only one thread runs Python bytecode at a time. Use processes to bypass it.
Race conditions	Shared data + concurrent writes = bugs. Use Locks.
Flynn's Taxonomy	SISD, SIMD, MISD, MIMD — MIMD is the most common today
Memory models	Shared memory (threads, multiprocessing) vs Distributed memory (MPI)
Foster's methodology	Decompose → Assign → Agglomerate → Map
Performance metrics	Speedup, Efficiency, Amdahl's Law (pessimistic), Gustafson's Law (optimistic)

Demo Programs Quick Reference

File	Functions (uncomment progressively)
<code>01_json_serialization.py</code>	<code>demo_basic_serialization</code> → <code>demo_list_serialization</code> → <code>demo_file_io</code> → <code>demo_limitations</code>
<code>02_pickle_serialization.py</code>	<code>demo_pickle_basic_types</code> → <code>demo_pickle_custom_objects</code> → <code>demo_pickle_file_io</code> → <code>demo_pickle_vs_json</code>
<code>03_threads_vs_processes.py</code>	<code>demo_threads_basics</code> → <code>demo_processes_basics</code> → <code>demo_gil</code> → <code>demo_race_condition</code>
<code>04_parallel_performance.py</code>	<code>demo_pool_map</code> → <code>demo_speedup_efficiency</code> → <code>demo_amdahl_gustafson</code>

What's Coming Tomorrow (Day 2)

Day 2 dives deeper into **shared-memory parallelism**:

- Advanced thread synchronization: Lock, RLock, Semaphore, Condition, Event
- Thread communication with queues
- Full process lifecycle: spawn, name, background, kill
- Inter-process communication: Queues, Pipes
- Shared state with `Value` and `Array`
- Process pools

End of Day 1